

Lab 16

File Handling

What is file handling in C++?

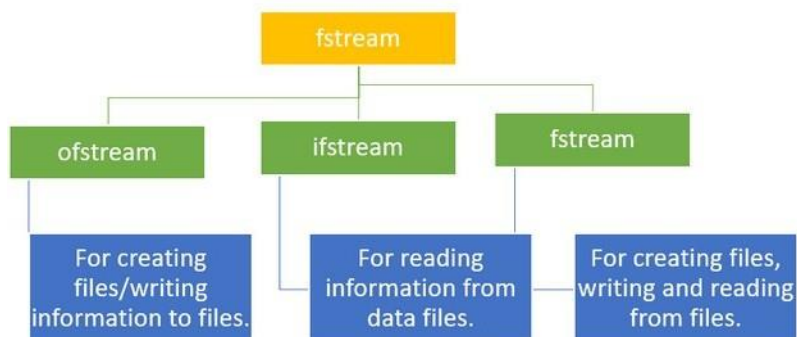
Files store data permanently in a storage device. With file handling, the output from a program can be stored in a file. Various operations can be performed on the data while in the file. A stream is an abstraction of a device where input/output operations are performed. You can represent a stream as either a destination or a source of characters of indefinite length. This will be determined by their usage. C++ provides you with a library that comes with methods for file handling.

The fstream Library

The fstream library provides C++ programmers with three classes for working with files. These classes include:

- **ofstream**- This class represents an output stream. It's used for creating files and writing information to files.
- **ifstream**- This class represents an input stream. It's used for reading information from data files.
- **fstream**- This class generally represents a file stream. It comes with ofstream/ifstream capabilities. This means it's capable of creating files, writing to files, reading from data files.

The following image makes it simple to understand:



To use the above classes of the fstream library, you must include it in your program as a header file.

How to Open Files

Before performing any operation on a file, you must first open it. If you need to write to the file, open it using ofstream or ifstream objects. If you only need to read from the file, open it using the ifstream object. The three objects, that is, ofstream, ifstream, and ifstream, have the open() function defined in them. The function takes this syntax:

```
open (file_name, mode);
```

- The file_name parameter denotes the name of the file to open.
- The mode parameter is optional. It can take any of the following values:

Value	Description
<code>ios::app</code>	The Append mode. The output sent to the file is appended to it.
<code>ios::ate</code>	It opens the file for the output then moves the read and write control to file's end.
<code>ios::in</code>	It opens the file for a read.
<code>ios::out</code>	It opens the file for a write.
<code>ios::trunc</code>	If a file exists, the file elements should be truncated prior to its opening.

How to Close Files

Once a C++ program terminates, it automatically

- flushes the streams
- releases the allocated memory
- closes opened files.

However, as a programmer, you should learn to close open files before the program terminates. The `fstream`, `ofstream`, and `ifstream` objects have the `close()` function for closing files. The function takes this syntax:

```
void close();
```

How to Write to Files

You can write to file right from your C++ program. You use stream insertion operator (`<<`) for this. The text to be written to the file should be enclosed within double-quotes.

```
#include <iostream>
#include <fstream>
using namespace std;
int main() {
    fstream my_file;
    my_file.open("my_file.txt", ios::out);
    if (!my_file) {
        cout << "File not created!";
    }
    else {
        cout << "File created successfully!";
        my_file << "Guru99";
        my_file.close();
    }
    return 0;
}
```

How to Write to Files character by character:

```
#include <iostream>
#include <fstream>

int main() {
    // Specify the file path
    const std::string filePath = "output.txt";

    // Open the file for writing
    std::ofstream outputFile(filePath);

    // Check if the file is open successfully
    if (!outputFile.is_open()) {
        std::cerr << "Error opening file: " << filePath << std::endl;
        return 1; // Return an error code
    }

    // Write a character to the file using put
    char ch = 'A';
    outputFile.put(ch);

    // Close the file
    outputFile.close();

    std::cout << "Character written to the file successfully!" << std::endl;

    return 0; // Return success code
}
```

How to Read from Files

You can read information from files into your C++ program using stream extraction operator (>>). You use the operator in the same way you use it to read user input from the keyboard. However, instead of using the cin object, you use the ifstream/ fstream object.

```

#include <iostream>
#include <fstream>
using namespace std;
int main() {
    fstream my_file;
    my_file.open("my_file.txt", ios::in);
    if (!my_file) {
        cout << "No such file";
    }
    else {
        char ch;

        while (1) {
            my_file >> ch;
            if (my_file.eof())
                break;

            cout << ch;

        }

    }
    my_file.close();
    return 0;
}

```

How to Read from Files Line by Line

```

#include <iostream>
#include <fstream>
#include <string>

int main() {
    // Specify the file path
    const std::string filePath = "input.txt";

    // Open the file for reading
    std::ifstream inputFile(filePath);

    // Check if the file is open successfully
    if (!inputFile.is_open()) {
        std::cerr << "Error opening file: " << filePath << std::endl;
        return 1; // Return an error code
    }

    // Read and print each line from the file
    std::string line;
    while (std::getline(inputFile, line)) {
        std::cout << line << std::endl;
    }

    // Close the file
    inputFile.close();
}

```

```
return 0; // Return success code
```

```
}
```

Tasks

Problem No: 1

Write a C++ program, which initializes a string variable to the content "**Time is a great teacher but unfortunately it kills all its pupils. Berlioz**" and write the string to a file OUT.txt. Then read the content from the text file OUT.txt, count and display the number of alphabets present in it.

Problem No: 2

Write a C++ program to create a file "**story.txt**". Write the following lines into the file using fstream.

"The rose is red.

A girl is playing there.

There is a playground.

An aeroplane is in the sky.

Numbers are not allowed in the password."

Now copy the contents of this file into another file named "**output.txt**"

Submission Instructions:

1. Save all .cpp files with your lab no, roll no and task number e.g. LAB14_i23XXXX_Problem01.cpp
2. Take the screenshots of your program outputs showing on the terminal.
3. Submit all .cpp files directly on Google Classroom.